Energy-Guided Sampling for Controllable Text Generation

From Langevin Dynamics to Amortized Inference

Sarthak Singh

Institut für Maschinelle Sprachverarbeitung, Universität Stuttgart

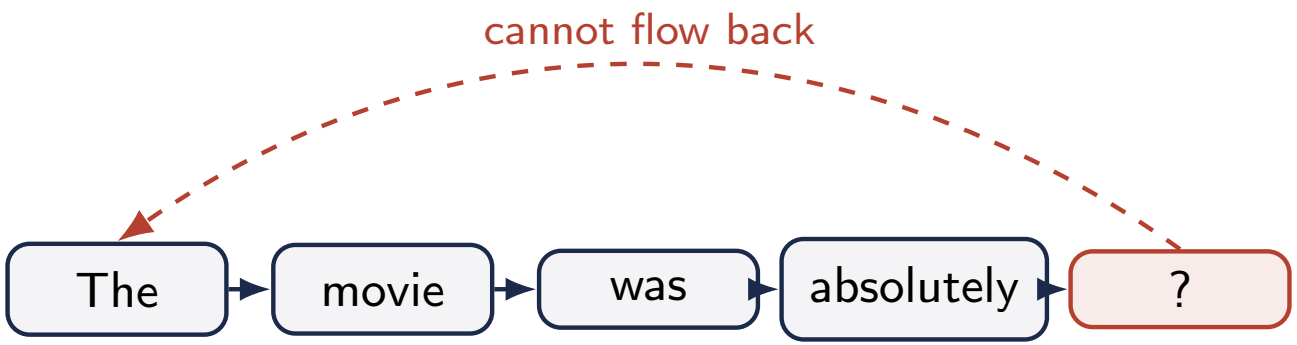
Examiners: Prof. Dr. Thang Vu Dr. Antje Schweitzer
Supervisor: Lukas Mauch

M.Sc. Computational Linguistics

Thesis Defense

The problem autoregressive models cannot solve natively

- Autoregressive generation: one token at a time, left to right.
- A token is fixed the moment it is emitted.
  No mechanism to revise an earlier token in light of a later one.
- Global properties (sentiment, factual constraint) live in the *whole* sequence.



Control means steering the *completed* object. That is a global problem.

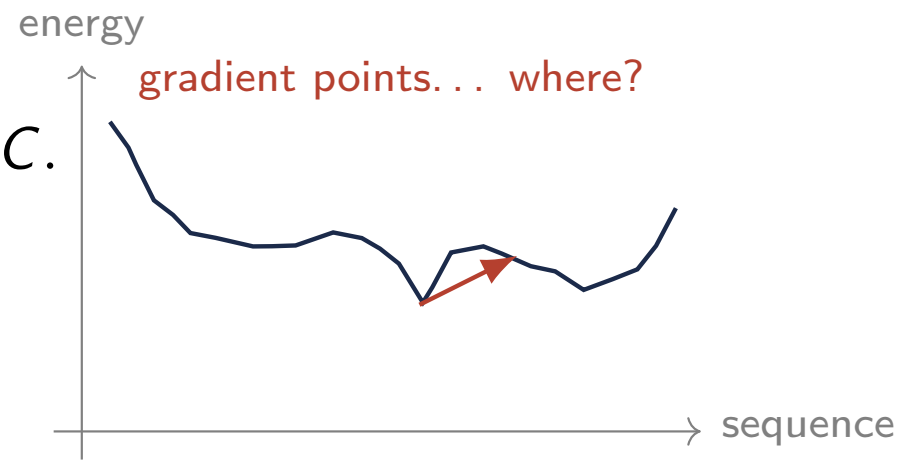
The energy-based promise, and its one assumption

Treat the frozen likelihood as an **energy** over sequences and sample from it:

$$p(\boldsymbol{x}) \propto \exp\big(-E(\boldsymbol{x})\big), \quad E = -\log p_{\mathrm{LM}} + \lambda\, C.$$

- Add any constraint C at inference time.
- One frozen model, any objective. **Plug and play.**

The untested assumption:
the gradient of the frozen likelihood is a usable search direction on discrete text.



A jagged landscape: the local slope need not lead anywhere good.

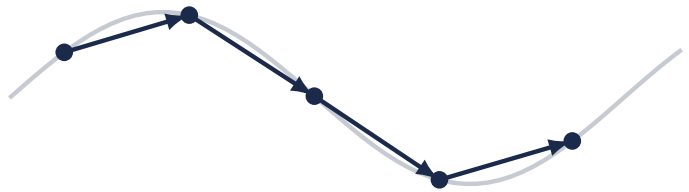
The machinery: Langevin dynamics

Sample $p(\boldsymbol{s}) \propto e^{-E(\boldsymbol{s})}$ with a gradient-guided chain:

$$\boldsymbol{s}_{t+1} = \boldsymbol{s}_t + \tfrac{\epsilon_t}{2} \nabla_{\boldsymbol{s}} \log p(\boldsymbol{s}_t) + \sqrt{\epsilon_t}\, \boldsymbol{\xi}_t$$

- **Drift**: uphill.
- **Diffusion**: a *sample*, not a mode.
- Needs a **Lipschitz** drift.

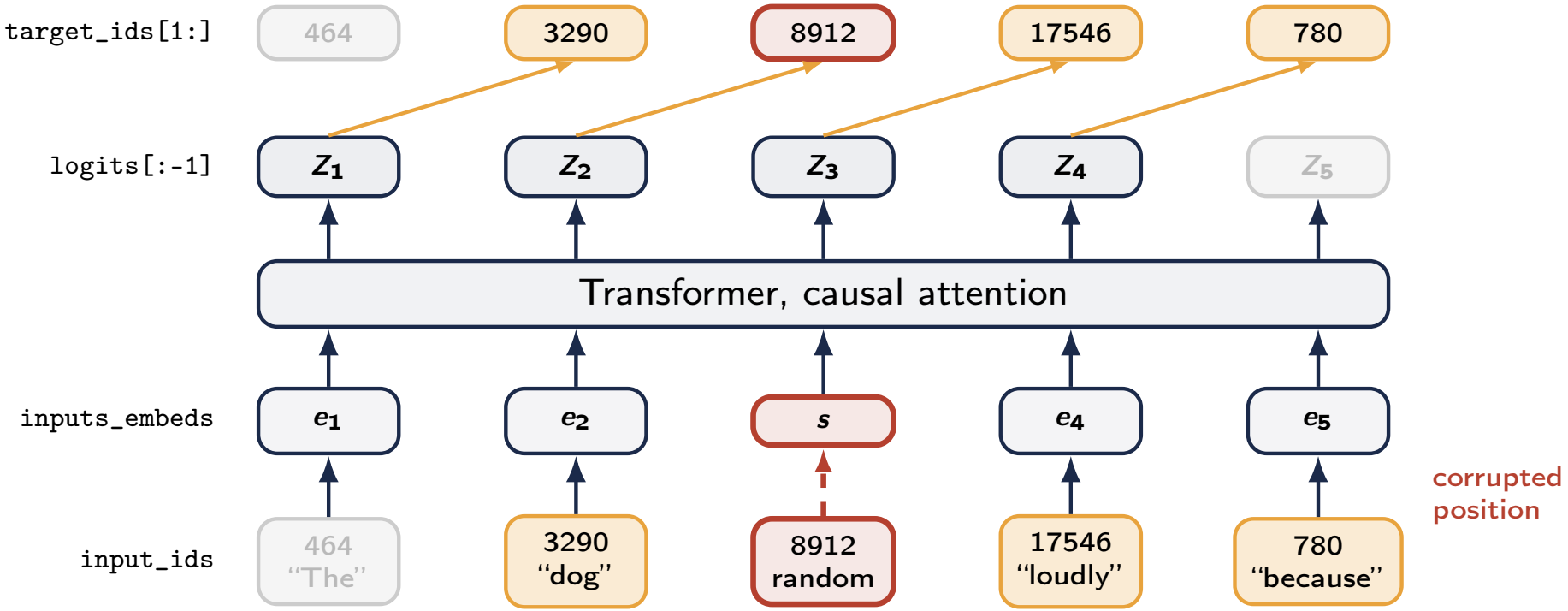
small steps, smooth surface



Text is *discrete*: there is no small step.

Every method here forces this update onto discrete tokens.

Scoring a sequence: two arrays, offset by one



$$E(\boldsymbol{x}) \;=\; -\sum_t \log \operatorname{softmax}(\boldsymbol{Z}_t)\big[\,\mathrm{id}_{t+1}\,\big]$$

The target row is the input row, shifted by one. The same sequence is present as continuous vectors and as integers.

Research questions

RQ1 Does the *input-embedding* gradient give a usable proposal direction? If not, what does?

RQ2 What does the Metropolis–Hastings correction do in each sampler?

RQ3a Does a GFlowNet on this reward learn a high-reward policy?

RQ3b Does its tuned energy become more navigable?

E (extension) Can an additive constraint steer generation here?

Two ways out, if the gradient fails. Repair the search (RQ1, RQ2), or abandon the search and *train a policy* to absorb it (RQ3). The talk takes them in that order.

Three candidate explanations

Suppose the gradient does not help. *Three* things could be at fault.

	The claim	Decided by
A target	No local search could profit from this likelihood. Then the proposal is beside the point.	search the same energy *without* a gradient
B model	Teacher forcing only ever asks for the next token, never to *rank a substitution*. Then only retraining repairs it.	re-differentiate the *same* frozen weights
C coordinates	The information is there and this derivative discards it. A proposal reading the *output side* recovers it; one seeing the *right context* recovers more.	a ladder of proposals in one exact chain

Everything that follows is that elimination, in that order.

The setup: one task, five energies, one metric

Task. Masked-token recovery: corrupt tokens, ask the sampler to put them back.

200 sequences per configuration, paired seeds across arms.

Metric. Mean KL to the ground-truth fill. Floor of 0.

Five energies.

- **GPT-2 Large**, fine-tuned. The reference.
- **Three GFlowNet-tuned** variants of it.
- **Llama-3 8B**. Cross-architecture.

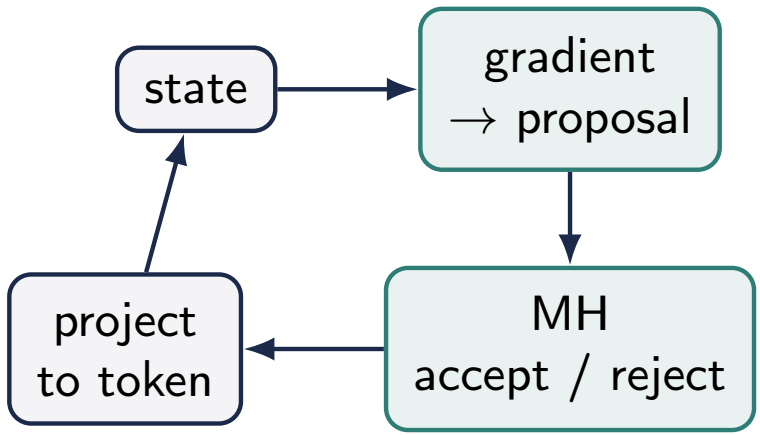
145 configurations. Equivalence margin fixed in advance at 0.327.

Two samplers, implemented faithfully

- **DLS** (discrete): stays in token space.
- **CLS** (continuous): relaxes into embedding space, projects back.
- **Metropolis–Hastings** correction, applied exactly.
- Step size by **oracle calibration**.

DLS scores a candidate v against x_i, with $\boldsymbol{g} = \nabla_{\boldsymbol{e}_i} \log p_{\mathrm{LM}}$:

$$\log q(v \mid x_i) \propto \underbrace{\tfrac{1}{2}\boldsymbol{g}^\top\big(\boldsymbol{e}(v) - \boldsymbol{e}(x_i)\big)}_{\text{gradient alignment}} - \underbrace{\tfrac{1}{2\alpha}\big\|\boldsymbol{e}(v) - \boldsymbol{e}(x_i)\big\|^2}_{\text{distance}}$$



the step the field skips

A fixed annealed schedule, never stopped early.

This $\boldsymbol{g}$, the input-embedding gradient, is the object under test.

First: what is the proposal we are about to ablate?

Entropy measures how spread out a distribution is, in *nats*.

- Concentrated on one token: 0.
- Flat over all 50,257: $\log |V| = 10.825$, the ceiling.

So entropy answers: *how many tokens is the proposal really choosing between?*

The calibrated proposal, measured

at its *sharpest*: 10.8248 nats
uniform ceiling: 10.8249 nats

Effectively choosing among $\approx$ 29,000 tokens at every step.

The gradient term spreads the logits by ≈ 0.009 nats. The softmax that consumes them has a 10.82-nat budget, so the gradient moves the proposal by about one part in a thousand.

The calibrated proposal is numerically uniform.

So a 5×5 sweep over step size and temperature re-tests everything below where the gradient *does* shape the proposal.

The central result: is the *direction* worth anything?

The control. Take the true gradient, throw away its direction, keep its length exactly. That is the *norm-matched random* arm. The two differ in direction and in nothing else.

Read the whole chain, not its end. Three summaries of the same run:

- **last step**: where it stopped.
- **chain mean**: its typical state.
- **best state**: the best it ever reached.

Policy minus norm-matched random, $n = 1000$ paired

last step	$+0.133$	$[-0.063, +0.326]$
chain mean	$+0.136$	$[-0.012, +0.287]$
best state	$+0.033$	$[-0.079, +0.145]$

Margin ± 0.327 nats, written down *before* the test: any gap smaller than this is too small to matter. All three intervals fit inside it.

So this is certified equivalence, not "we failed to find a difference".

Same picture on all five energies, and in the sweep recovery never exceeds 2% in any of the 25 cells.

Hypothesis A: maybe the energy itself is hopeless?

Then try something simpler: search the *same* energy without ever differentiating it.

Method (no gradient of the energy)	KL	exact
Ground-truth fill (floor)	0.00	—
Corrupted token left in place	9.14	0%
Best Langevin cell *(uses the gradient)*	~ 6.4	0.0%
Gradient-free Gibbs sampler	6.69	18.5%
Top-k rescore, one forward pass	4.43	33%

Gibbs: blank one position, ask the model for its distribution there, draw a token, move on. Repeat. No gradient anywhere.

A is rejected: the energy is searchable, just not by this derivative.

And the cheap methods are the good ones: no backward pass at all.

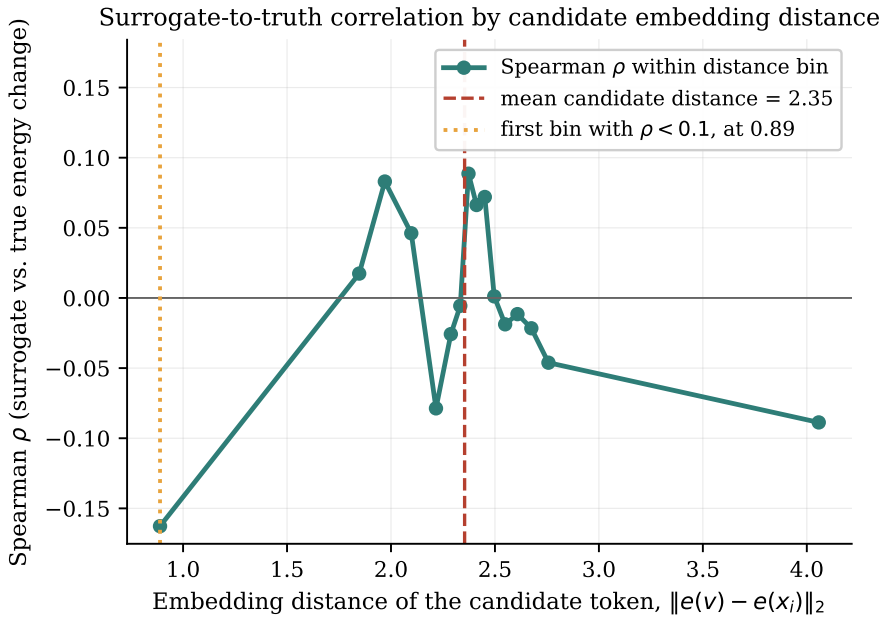
Why, part 1: the approximation dies before the first step

The proposal ranks a candidate by a straight-line estimate of what swapping it in would do:

$$\hat{\Delta}(v) = \boldsymbol{g}^{\top}\big(\boldsymbol{e}(v) - \boldsymbol{e}(x_i)\big)$$

A straight line only fits a curve *near* the point it is drawn at. So: how far does it stay accurate?

Measured: correlate the estimate against the truth, *within bins of distance*.

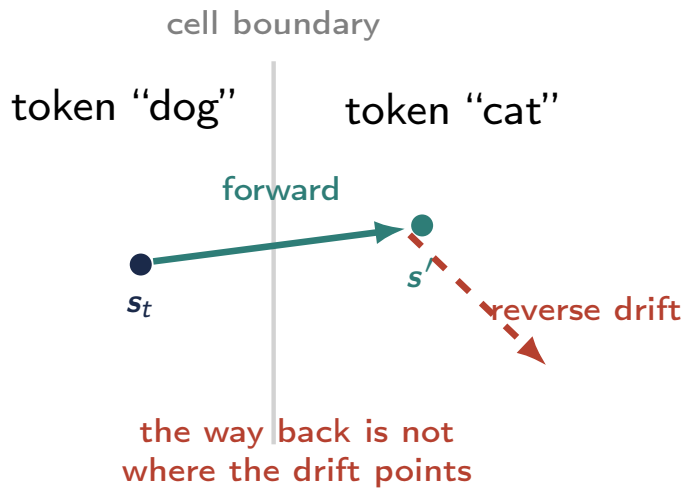


Correlation with the true change, by how far the swap moves you.

It is at zero by distance 1.82, which is the average gap between *neighbouring* tokens: the smallest move that changes any word at all.

Every legal move lies outside the range where the estimate works.

RQ2: what does the accept/reject step actually do?



Cross a boundary and the projected token changes, so the drift on the far side is computed for a *different* prediction. The reverse move is not proposable.

The acceptance ratio is a product of two things:

$$\alpha = \underbrace{\frac{\pi(s')}{\pi(s_t)}}_{\text{is it better?}} \cdot \underbrace{\frac{q(s_t \mid s')}{q(s' \mid s_t)}}_{\text{could we come back?}}$$

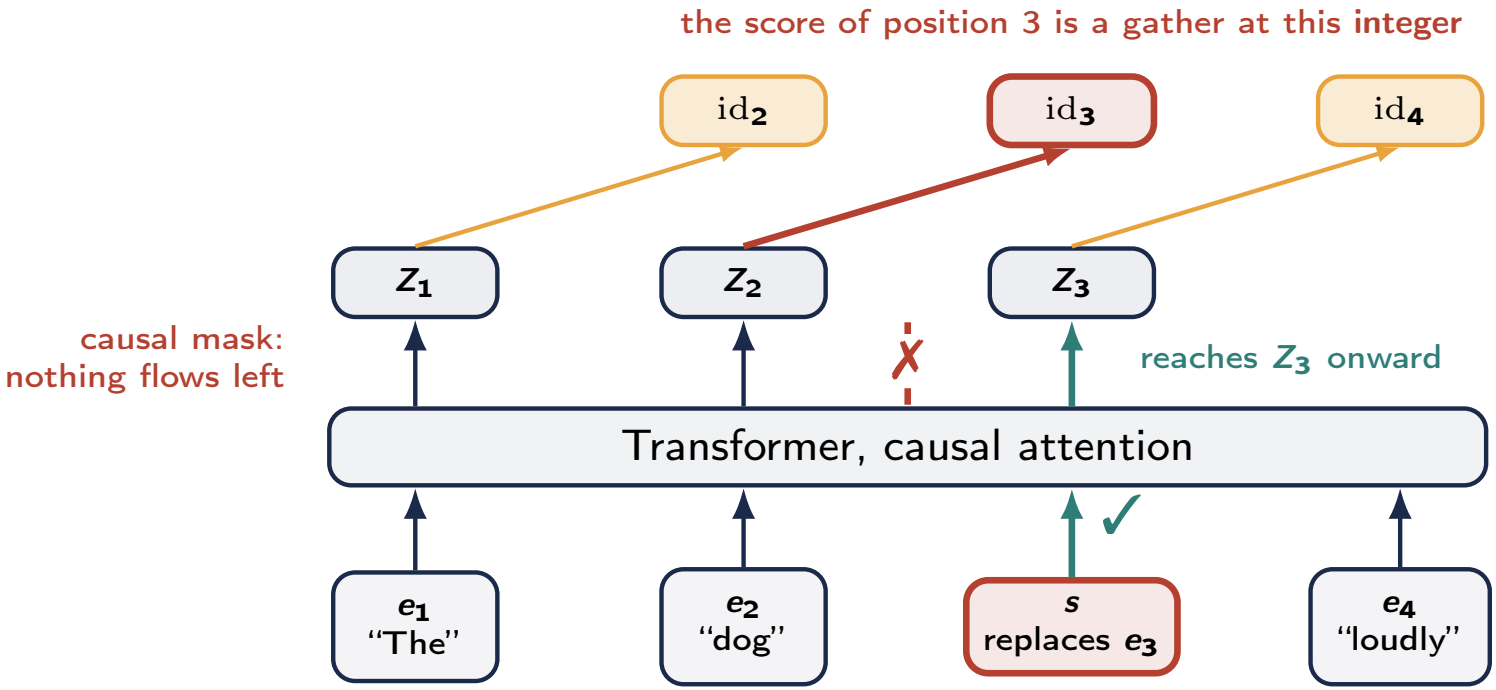
Boundary-crossing moves, measured

"is it better?" $= e^{+4.60}$ yes
"could we come back?" $= e^{-1325}$

The second term is 10^{575} times smaller. Good moves are rejected for being *unreturnable*, not for being bad.

Continuous: disabled (3.7–8.6%). Discrete: rescued (100% within-cell).

Back to the two arrays: what the gradient cannot reach



$\boldsymbol{Z}_2$ was built from $\boldsymbol{e}_1, \boldsymbol{e}_2$ alone, and id_3 is the *integer* $\boldsymbol{s}$ stands in for. So that derivative is exactly 0.

future term, what it had
$\boldsymbol{g}^\top (\boldsymbol{e}(v) - \boldsymbol{e}(x_i))$

$+$

self term, read off the logits
$\log p(v \mid \boldsymbol{x}_{<i})$

Hypothesis B: same model, same sampler, different derivative

Relax the token indicator $\boldsymbol{z}_i$ in *both* of its roles:

$$\frac{\partial \tilde{L}}{\partial \boldsymbol{z}_i[v]} = \underbrace{\log p(v \mid \boldsymbol{x}_{<i})}_{\text{self, discarded}} + \underbrace{\boldsymbol{g}^\top \boldsymbol{e}(v)}_{\text{future}}$$

Same weights, same sampler, same energy. Only the derivative changes.

ρ (near): how well the estimate *ranks* candidates against the truth, over the swaps that are actually reachable. 1 = perfect order, 0 = no order.

Surrogate	ρ (near)	exact
input embedding	0.03	0.0%
token indicator	0.73	40.0%

Llama-3 8B reaches 41.0% where every input-embedding run gives 0.0%.

$$0 \rightarrow 40\%$$

B rejected in its strong form

A clean confirmation: at the last position the gradient is provably zero

The final token's embedding feeds no prediction that exists, so

$$\nabla_{\boldsymbol{e}_L} \log p_{\mathrm{LM}}(\boldsymbol{x}) = 0 \quad \text{(exactly)},$$

while the energy *difference* between candidate final tokens is exactly the model's conditional log-ratio.

Energy maximally usable; gradient provably useless.

Position	LM grad norm
final token	0.0000
second-to-last	> 0
middle	> 0

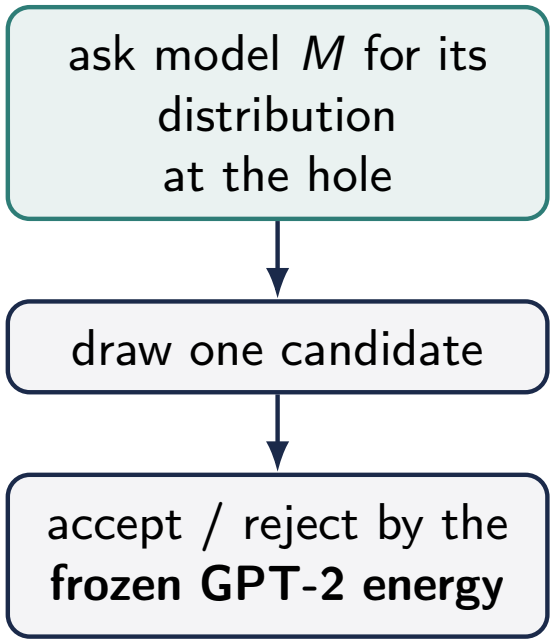
Langevin recovers 0% at all three; exact-energy methods 18–55%.

Hypothesis C: swap the proposal, keep everything else

An autoregressive model can only ever look left. To fill a hole it has no path from the words *after* it.

Two model families can look both ways:

- **SEDD**, a discrete diffusion model: re-mask a position, re-denoise it from both sides.
- **RoBERTa**, an ordinary masked LM: the same query, and its native training task.



An independence sampler: *M* only *suggests*. The energy being sampled is still GPT-2's, unchanged. Same 200 sequences, same corruptions, same metric.

Only the box marked *M* changes. So whatever moves is caused by the proposal.

The conditioning ladder

Proposal	can see	exact
input-emb. gradient	nothing	0.0%
norm-matched random	nothing	0.0%
uniform draw	nothing	0.5%
GPT-2's own conditional	left only	23.5%
SEDD (score-trained)	both sides	39.0%
RoBERTa MLM (no score obj.)	both sides	44.5%

The uniform draw reproduces the flagship Langevin sampler, KL 6.538 vs 6.541, from separate code. So the pipeline leaks nothing.

Two claims, both confirmed

Output side beats no side:
$0.5 \rightarrow 23.5$.

Both sides beat left only:
$23.5 \rightarrow 44.5$.

Gradient-free comparator on the same task: top-k rescore 33%. So 44.5 is a gain over 33, not over nothing.

What orders this is access to context, not the training objective.

GFlowNet amortization does not escape the failure

A GFlowNet only ever *evaluates* the reward, never differentiates it.

- Three failure modes: **capacity collapse**, **length collapse**, **reward hacking**.
- Tuning moved the energy by 31–57 nats...
- ...yet Langevin sampling on the tuned energy is *indistinguishable* from the untuned base.

Changing the model through this objective did not make the local surrogate useful.

The extension: can we actually steer it?

The original promise: $E = -\log p_{\text{LM}} + \lambda C$, with C a sentiment classifier. Does adding C move the output?

Sampler	Target	Steering
MuCoLa-style continuous	negative	+27.3
MuCoLa-style continuous	positive	+36.7
Discrete sampler (ours)	negative	0.0
Discrete sampler (ours)	positive	+0.7

Percentage points, *constraint only* minus *randomized constraint*. That subtraction cancels a large sentiment drift present in every arm, including the ones with no constraint at all.

How to read this

The constraint gradient's direction carries signal where the fluency gradient's never did.

But the arm that steers has no fluency term holding it down, so it drifts off the manifold of real text and the classifier scores something it was never trained on.

A constraint added to a landscape no sampler can navigate is added to nothing.

So put the constraint on a landscape that *is* navigable

Carry the constraint on the bidirectional proposal instead: train a classifier to read the diffusion model's partially masked states, and reweight its candidates at each commitment.

The classifier that *guides* and the judge that *grades* are separate models throughout.

- Steering works: $+9.3$ to $+25.7$ points, both labels, every strength.
- It steers its own classifier more reliably than the independent judge.
- Transfer is bounded by how far the two instruments *agree*: $56\% \rightarrow 80\%$.
- A trust region removes the fluency cost ($7.1 \rightarrow 11.0$ nats without it).

Control follows navigation. Explored, not settled: a repair on *another* landscape, not an answer about the autoregressive energy.

Takeaways

1. The direction is worthless, and that is certified. Replacing the gradient with a random direction of the same length changes nothing, inside a margin fixed in advance, over 1000 paired sequences.
2. The energy is fine. The model is fine. The *coordinates* were wrong. A derivative of the same frozen weights, taken with respect to the token's identity rather than its embedding, goes from 0% to 40%.
3. The rest is about seeing both sides. A plain masked language model, never trained on scores, beats a purpose-built diffusion model. Access, not objective.
4. Control follows navigation. A constraint buys nothing where the proposal cannot move, and buys a lot where it can.

Differentiate the right object, or propose from the output side.

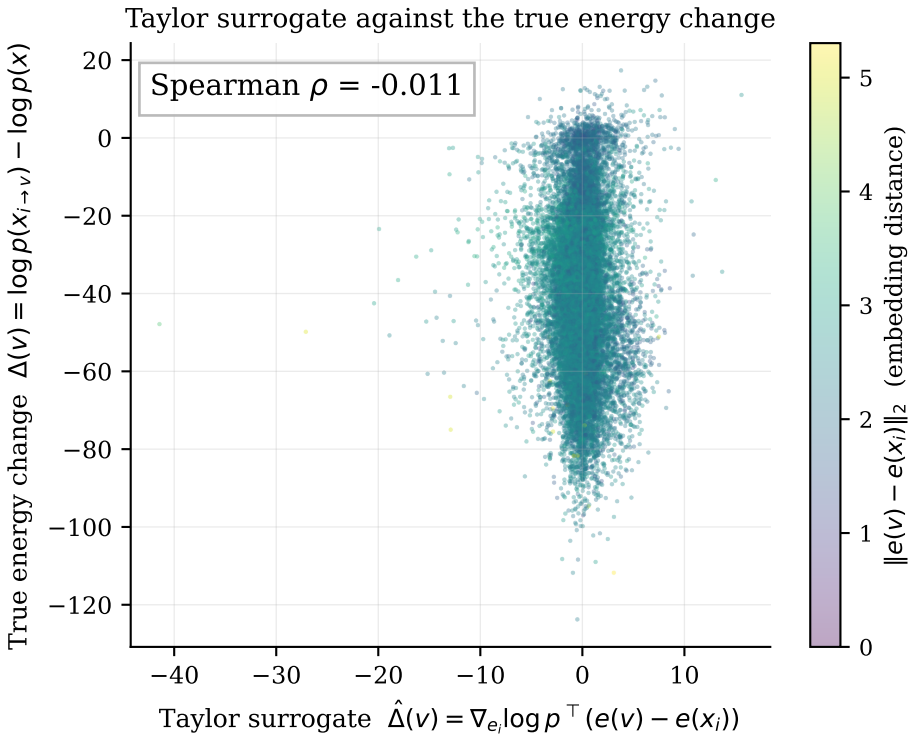
Not: evaluate rather than differentiate. The derivative was in the wrong coordinates.

Thank you.

Questions?

Backup: the linearization radius and self-term blindness

- The surrogate correlates with the *future* term ($\rho = 0.03$), never the *self* term ($\rho = -0.10$).
- Mean absolute self term 15.0 nats; future term 24.2 nats.
- The input-embedding gradient is structurally blind to the candidate's own left-context fit, which enters as a discrete index.



Backup: the Metropolis–Hastings chain

$$\alpha = \min\left\{1,\ \frac{p(\boldsymbol{s}')\, q(\boldsymbol{s} \mid \boldsymbol{s}')}{p(\boldsymbol{s})\, q(\boldsymbol{s}' \mid \boldsymbol{s})}\right\}$$

- Target ratio: prefers good states. Proposal ratio: enforces reversibility.
- MALA needs a Lipschitz drift (Roberts and Tweedie 1996; Dalalyan 2017; Durmus and Moulines 2017).
- The projected target is piecewise constant; the differentiable pathway's drift jumps across a cell boundary, so the reverse density vanishes.

Backup: what the sampler does without the correction

- Correction off: a uniform random walk that never localizes. Mean KL *rises* from 8.765 to 9.499; the chain still changes its token on 99.1% of final-decile steps.
- So the late entropy dip is the annealing floor, not a quench: no freezing, no convergence to a local optimum.
- Correction on: the same uniform proposal becomes an independence sampler, the accept/reject filter does all the work, final KL 6.541 at a 9.98% acceptance rate.

Backup: likelihood is a poor objective, yet used everywhere

- The **likelihood trap**: the lowest-energy text is the most degenerate (Holtzman 2020, reproduced here). Within-strategy repetition correlation up to $+0.91$.
- Resolution: likelihood ranks single-token substitutions well (used by the KL metric and Gibbs), but maximizing absolute sequence likelihood in free generation is degenerate.
- So using it as the KL metric, the reward, and the SEDD comparison is consistent: relative-to-reference fit, not absolute likelihood as quality.

Backup: MuCoLa / COLD, and why they appear to work

- The continuous sampler follows their state *geometry* faithfully, a continuous embedding state with a projection after every update, and adds the correction those works omit. It does *not* follow their *energy*: see backup 33.
- Without the exact correction the update, *as reimplemented and measured here*, behaves as an early-stopped biased optimizer rather than a sampler from the stated energy. That is a statement about the mechanism, not a reinterpretation of those systems' published results, whose tasks, tuning and evaluation this study did not reproduce.
- The constrained *mucola* arm is that comparison, run at the energy level (final KL under the same base model), not the task level.

Backup: task design and corpus

- Masked-token recovery on WikiText-2 validation; the base GPT-2 Large is fine-tuned on ROCStories (out-of-domain test).
- In-domain ROCStories diagnostics reproduce the null and the likelihood trap: not a domain-shift artifact.
- The KL metric compares the recovered token's next-token conditional against the ground-truth-conditioned reference; synonyms score well.
- Llama-3 is a cross-architecture control, never on a shared numerical axis. One Llama contrast is nominally significant against the gradient, but it is one of dozens uncorrected, is measured with the correction off, and does not reproduce on GPT-2 sharp cells; the supported claim is *indifference*, not anti-guidance.

Backup: why did the temperature flatten everything?

- Proposal logit $= \big[\underbrace{\tfrac{1}{2}\boldsymbol{g}^\top(\boldsymbol{e}(v) - \boldsymbol{e}(x_i))}_{t_2} - \underbrace{\tfrac{1}{2\alpha}\|\boldsymbol{e}(v) - \boldsymbol{e}(x_i)\|^2}_{t_1} \big] / T.$
- At the calibrated cell $T = 5$ divides a spread of a few nats into a 10.82-nat entropy budget: the softmax is uniform whatever the surrogate says.
- Sharpness from the *distance* term is sharp about *staying put* ($t_2/t_1 = 1.8$); sharpness from the *surrogate* is sharp about the ranking ($t_2/t_1 = 229$). Only the second tests the direction.
- The recovering cell needs large ϵ (kills t_1) *and* low T together.

Backup: what exactly is the token-indicator derivative?

Relax the token indicator $\boldsymbol{z}_i$ from its simplex vertex, in *both* roles:

$$\tilde{L}(\boldsymbol{z}_i) = \sum_v \boldsymbol{z}_i[v] \log p(v \mid \boldsymbol{x}_{<i}) \; + \; L_{\text{future}}(\boldsymbol{E}^\top \boldsymbol{z}_i)$$

- First term: the current position's own log-likelihood with the discrete *target* relaxed. Second term: the suffix, with the relaxed indicator mapped through $\boldsymbol{E}$ to an input embedding.
- At a vertex $\tilde{L}$ equals the sequence log-likelihood, so this is a relaxation of the same object.
- It is *not* $\boldsymbol{E}^\top \boldsymbol{g}$, the ordinary one-hot input gradient through the embedding lookup: that carries the future term only. The whole difference is the self term.

Backup: why does RoBERTa beat SEDD?

- Both are bidirectional; only SEDD is score-trained. RoBERTa 44.5% vs SEDD-medium 39.0%, so the score objective is not what buys recovery.
- RoBERTa-large is bigger and trained on more text than SEDD-small/medium, and masked-token prediction *is* its native query at any noise level, whereas SEDD answers it at a schedule point.
- So the claim is an *ordering by conditioning*, not an isolation of it and not a ranking of the two models: the arms differ in scale, corpus and architecture as well, and only the tokenizer and the chain are held fixed across all of them. The SEDD arms are pilots.
- Tokenizer bridge: 99.95% of RoBERTa ids map to one GPT-2 token; no ground-truth token fell outside it. Uniform control through the same code: 0.5%, so the pipeline leaks nothing.

Backup: why a GFlowNet, and the task-comparability caveat

- A GFlowNet samples in proportion to reward (not the mode), and never differentiates it, so a gradient pathology would be invisible to it.
- It reformulates infilling as left-to-right generation under a restructured prompt; the samplers do in-place bidirectional substitution.
- The comparison is therefore run at the energy level, not the task level; the left-to-right reformulation is not adopted for the samplers because it would dissolve the revision capability under study.

Backup: how this differs from MuCoLa and COLD

Method	how the token's *own* score enters	self term
MuCoLa	softmax numerator $h_n^\top \tilde{\boldsymbol{e}}_{n+1}$, target relaxed	kept
COLD	$\sum_v p_{\mathrm{LM}}(v \mid \tilde{y}_{<t}) \log \mathrm{softmax}(\tilde{y}_t(v))$	kept
CLS, DLS (this thesis)	gather at a hard, projected index	discarded
token indicator (ours)	$\log p(v \mid \boldsymbol{x}_{<i})$, indicator relaxed	kept

- No, they do not share the blindness. Both relax the *target* as well as the input, which is exactly the design that makes the self term differentiable. MuCoLa says so explicitly: $\tilde{\boldsymbol{e}}_{n+1}$ receives gradients "directly from $-\log P$" as well as through the later hidden states.
- The blind object is mine. This implementation scores the masked position by gathering at the projected token id, so $\nabla_{\boldsymbol{e}_i}$ reaches the suffix only. The null is about *that* energy in embedding coordinates.
- And the repair converges with theirs. MuCoLa's self-gradient is h_n, ranking candidates by $h_n^\top(\boldsymbol{e}(v) - \boldsymbol{e}(x_i))$, the logit difference. The token-indicator self term is $\log p(v \mid \boldsymbol{x}_{<i}) - \log p(x_i \mid \boldsymbol{x}_{<i})$. Those agree up to the shared denominator, which cancels. The term worth $0 \to 40\%$ is the one MuCoLa already had.
- What was reimplemented faithfully is their sampler *geometry*: continuous embedding state, nearest-neighbour projection, plus the correction they omit. Not their energy.